

Diagram A — Identity Domain: Core Tables

users (root) → credentials | sessions | service_accounts → api_keys | identity_realms

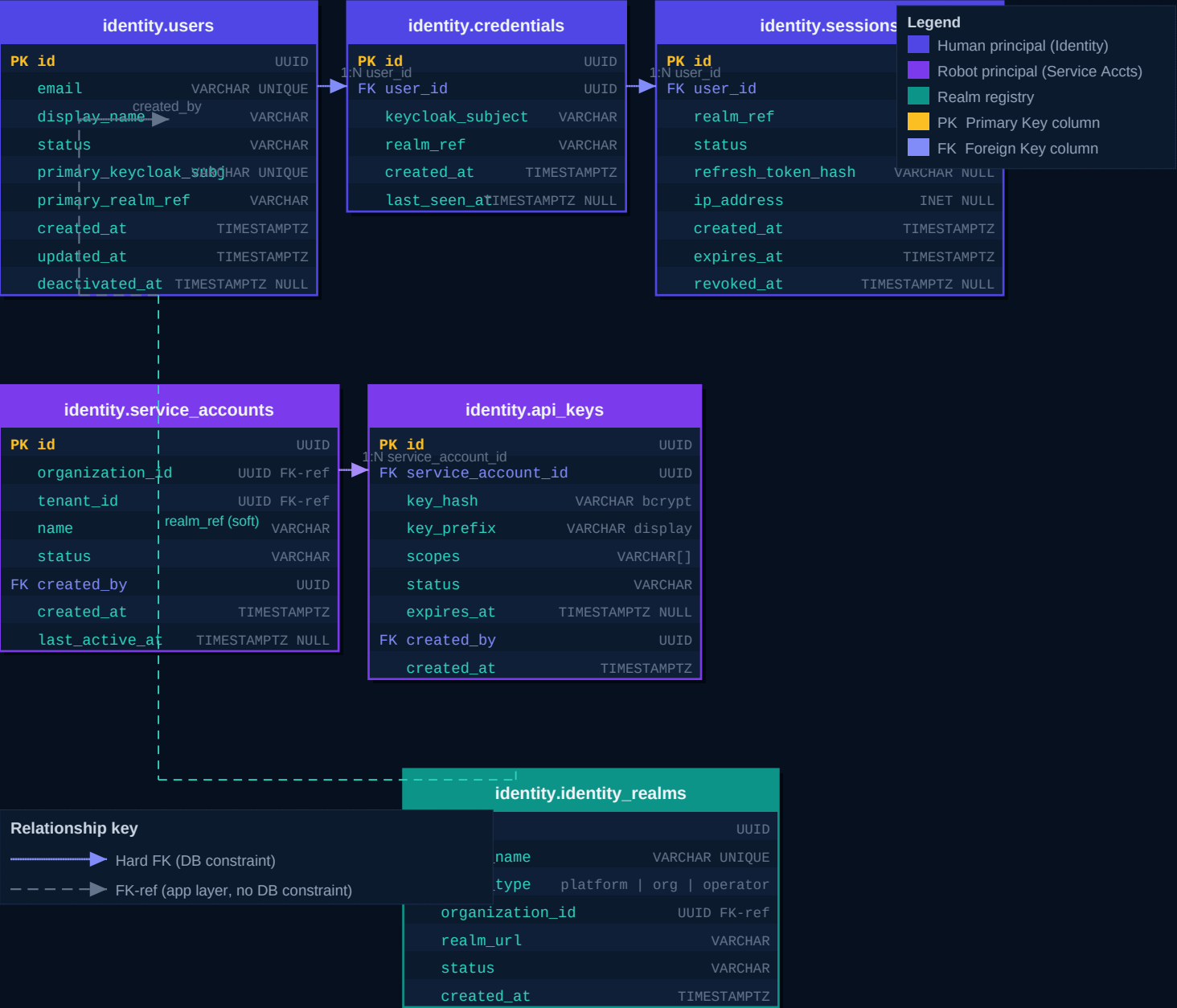


Diagram B — Keycloak Realm Types & Binding

Three realm types: platform (shared) · org (per company) · operator (VPN/admin only)

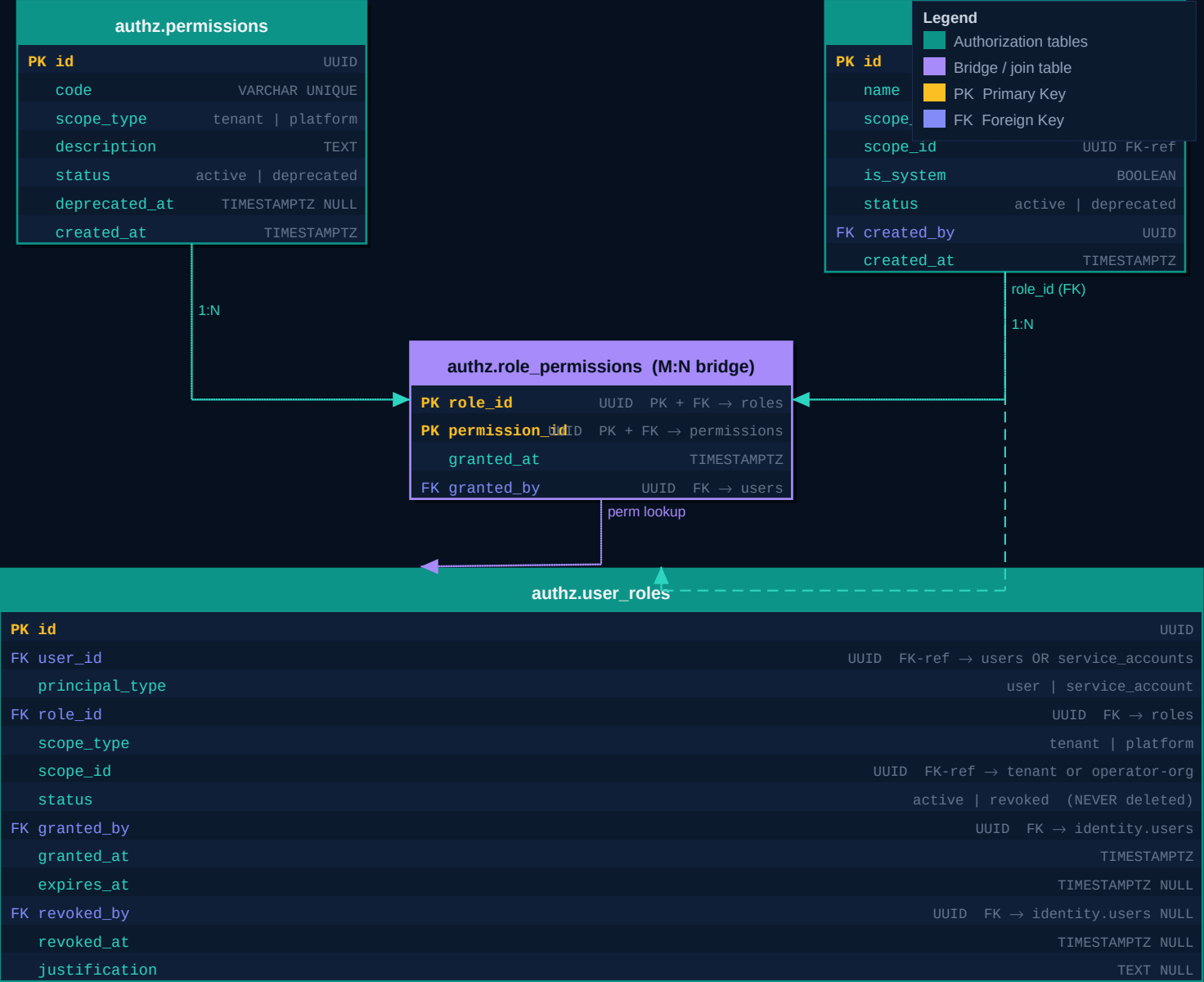


How realm assignment works:

1. On signup: users.primary_realm_ref = 'platform' (shared realm, created once at deployment)
2. On org upgrade: identity.identity_realms row created for 'acme-corp'
3. organizations.keycloak_realm_ref = 'acme-corp' stored permanently
4. All org team members log in via acme.auth.platform.io (their dedicated realm)
5. JWT 'iss' claim = realm URL. Admin API middleware checks: iss must = operator realm
6. Customer tokens (iss = platform or org realm) are REJECTED on admin routes — blocked before DB

Diagram C — Authorization Domain: 4 RBAC Tables

permissions → role_permissions ← roles and roles → user_roles



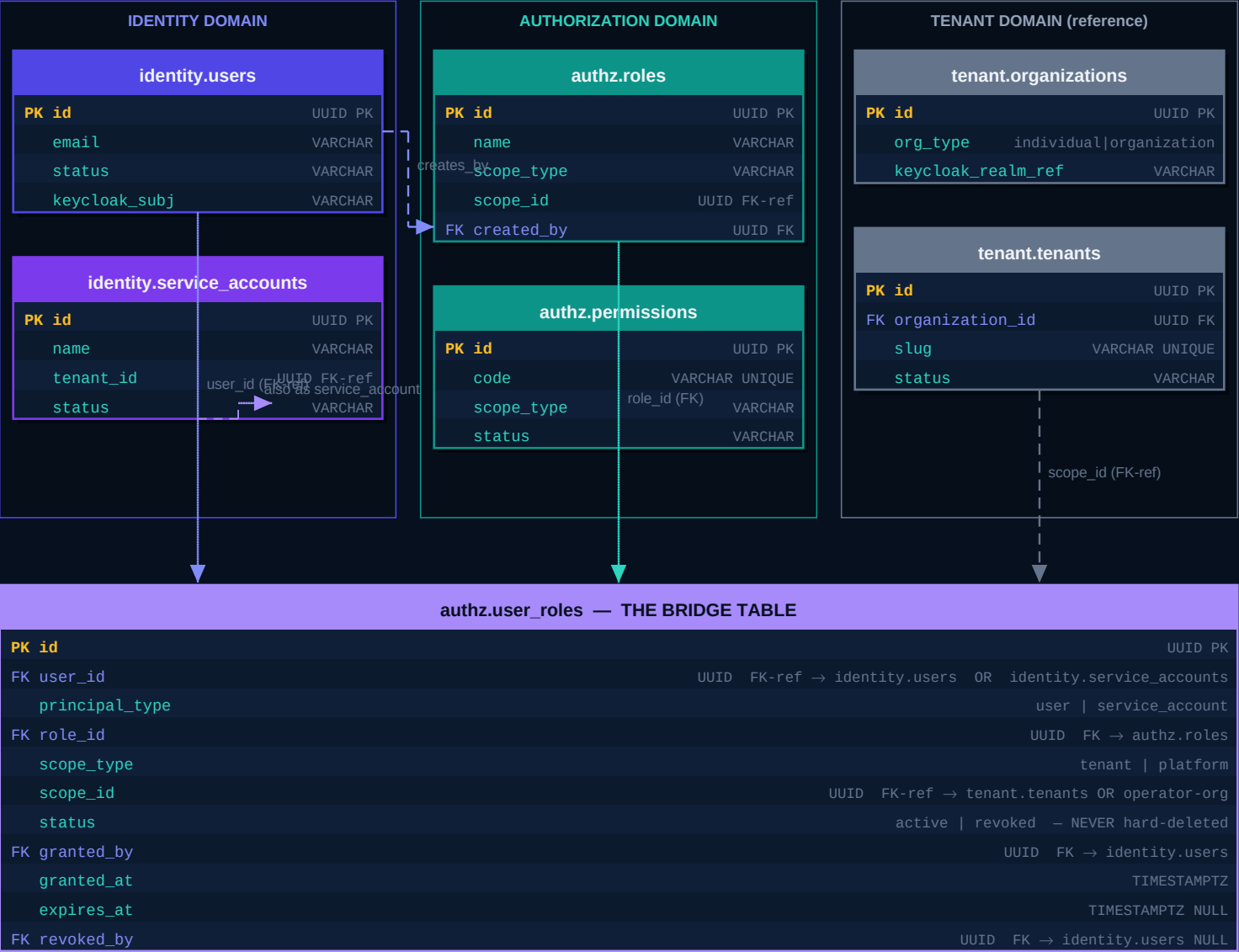
Critical INDEX (hot path — queried on EVERY API request):
CREATE INDEX ON authz.user_roles(user_id, scope_type, scope_id, status);
SELECT p.code FROM user_roles ur JOIN role_permissions rp ON ur.role_id=rp.role_id
JOIN permissions p ON rp.permission_id=p.id
WHERE ur.user_id=? AND ur.scope_type='tenant' AND ur.scope_id=? AND ur.status='active' AND p.code=?

Relationship key

- Hard FK (DB constraint)
- FK-ref (app layer, no DB constraint)

Diagram D — Cross-Domain: Identity → Authorization → Tenant

authz.user_roles bridges identity principals to roles, with scope pointing to Tenant domain



Key design notes:

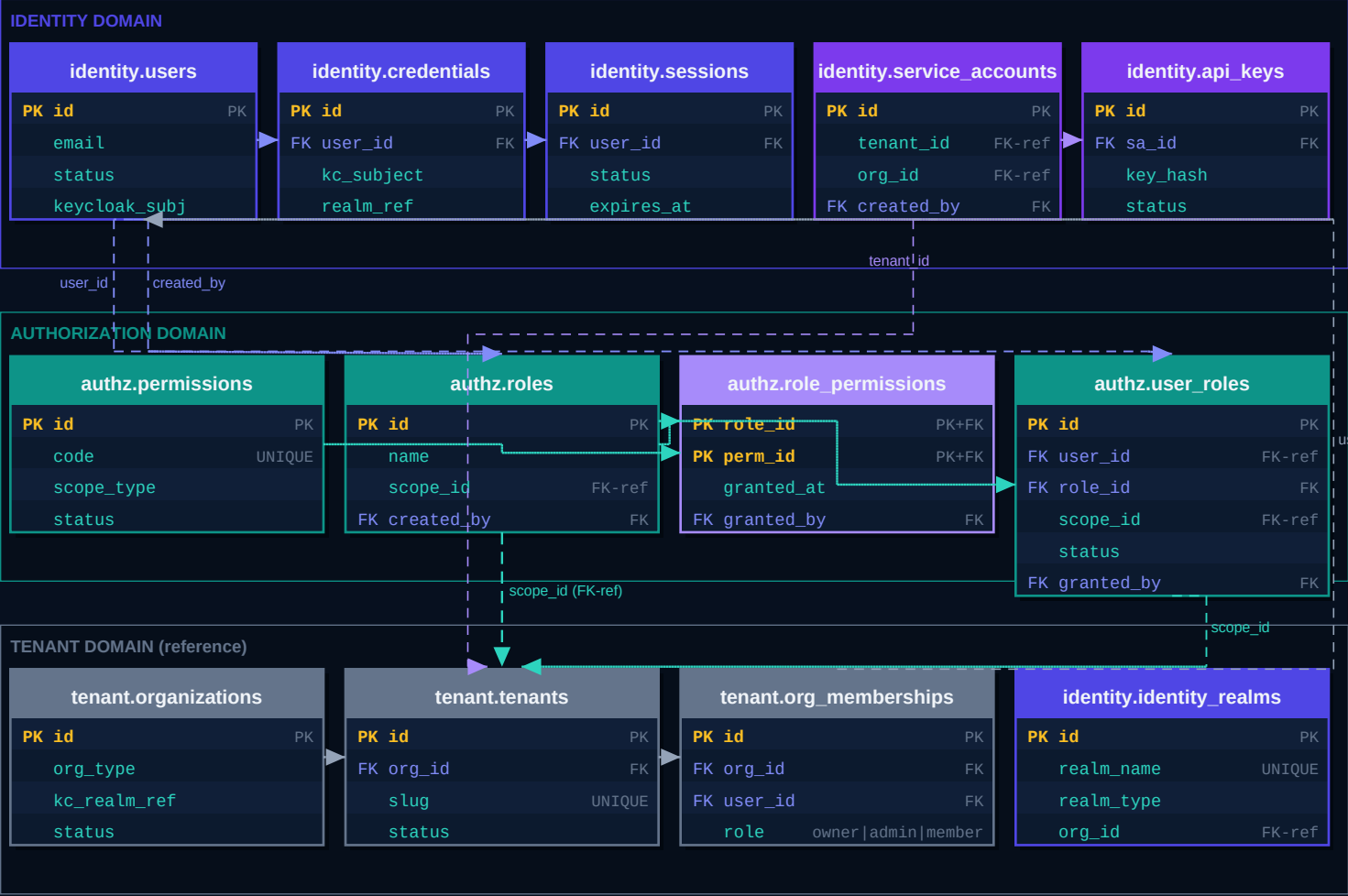
user_roles.user_id is POLYMORPHIC — references identity.users OR identity.service_accounts.
The principal_type column ('user' or 'service_account') tells the app which table to look in.
user_roles is NEVER hard-deleted. Revoked rows (status='revoked') stay for audit history.
Example: 'Did alice have admin access at 09:58 on 15-Mar?' needs the historical row.
scope_id is a FK-ref (not hard FK) to preserve domain independence between authz and tenant schemas.
authz.permissions is the Authorization schema.

Relationship key

- ▶ Hard FK (DB constraint)
- - - -▶ FK-ref (app layer, no DB constraint)

Diagram E — Full Platform Overview (Both Domains + Tenant)

Complete picture from user registration through role assignment to tenant resource access



Relationship key

- Hard FK (DB constraint)
- FK-ref (app layer, no DB constraint)

Legend

- Identity tables
- Service Account tables
- Authorization tables
- Bridge / join table
- Tenant domain (ref)

Identity & Authorization

Complete Database Schema Reference

Multi-Tenant Cloud Platform — Phase 1 & Phase 2

Pages 1–5 contain ERD diagrams. Pages 6 onward contain table DDL, FK sheet, and explanations.

Page(s)	Section
1	Diagram A — Identity Core Tables
2	Diagram B — Keycloak Realm Binding
3	Diagram C — Authorization Core (4 RBAC tables)
4	Diagram D — Cross-Domain Bridge (Identity → Authorization → Tenant)
5	Diagram E — Full Platform Overview
6	Cover & Table of Contents (this page)
7	Domain Overview
8–9	Identity Domain — all 7 tables
10–11	Authorization Domain — all 4 tables
12	FK / Reference Sheet
13–14	Schema Design Decisions & Explanations

Column colour coding: **AMBER** = Primary Key · **INDIGO** = Foreign Key · **TEAL** = Regular column

Domain Overview

Tables belong to exactly one domain. Cross-domain references use stored UUIDs (FK-refs) — never hard foreign keys across domain boundaries.

Domain	Tables	Purpose	Schema
Identity	users, credentials, sessions, service_accounts, api_keys, identity_realms, audit_log	Who is this principal?	identity.*
Authorization	permissions, roles, role_permissions, user_roles	What can this principal do?	authz.*
Tenant	organizations, org_memberships, tenants, projects, tenant_memberships	Where does this principal work?	tenant.*
Resource	resource_types, quota_policies, allocation_requests, resource_instances	What resources does the tenant have?	resource.*
Billing	plans, subscriptions, invoices	What is the tenant paying?	billing.*

This document covers Identity and Authorization only.

Identity Domain — 7 Tables

Answers: "Who is this person and how do we prove it?" Schema: identity

identity.users

Root human principal record. Status machine: pending_verification → active → locked → deactivated (terminal).

Column	Type	Constraints	Description
id	UUID	PK DEFAULT gen_random_uuid()	Stable forever — never changes.
email	VARCHAR(255)	UNIQUE NOT NULL	Login identity. Unique across entire platform.
display_name	VARCHAR(255)	NOT NULL	Human-readable name for UI.
status	VARCHAR(30)	NOT NULL DEFAULT 'pending_verification'	pending_verification active locked deactivated (terminal)
primary_keycloak_subject	VARCHAR(255)	UNIQUE NOT NULL	Matches JWT sub claim from Keycloak.
primary_realm_ref	VARCHAR(100)	NOT NULL	Soft link to identity_realms.realm_name.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Registration timestamp.
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Auto-updated by trigger.
deactivated_at	TIMESTAMPTZ	NULL	Set on deactivation. Kept forever for audit.
UNIQUE(email) · UNIQUE(primary_keycloak_subject) · INDEX(status)			

identity.credentials

Maps Keycloak kc_subject → platform user_id. One row per (user, realm) pair.

Column	Type	Constraints	Description
id	UUID	PK	
user_id	UUID	NOT NULL FK → users(id) CASCADE	Platform user. Deleted when user is purged.
keycloak_subject	VARCHAR(255)	NOT NULL	The kc_subject from JWT sub claim.
realm_ref	VARCHAR(100)	NOT NULL	Which Keycloak realm issued this credential.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	When credential was registered.
last_seen_at	TIMESTAMPTZ	NULL	Updated on every JWT validation.
UNIQUE(keycloak_subject, realm_ref) · INDEX(user_id)			

identity.sessions

Active login sessions. Revoking invalidates the refresh token immediately (next access check = DENY within seconds).

Column	Type	Constraints	Description
id	UUID	PK	Session UUID — matches JWT sid claim.
user_id	UUID	NOT NULL FK → users(id) CASCADE	Session owner.
realm_ref	VARCHAR(100)	NOT NULL	Which realm issued the session.
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active revoked expired
refresh_token_hash	VARCHAR(255)	NULL	bcrypt hash. NULL until first refresh.

identity.service_accounts

Robot principals for automation. Scoped to exactly one tenant + org — cannot act outside that scope.

Column	Type	Constraints	Description
id	UUID	PK	
organization_id	UUID	NOT NULL FK-ref → tenant.organizations	Owning organization.
tenant_id	UUID	NOT NULL FK-ref → tenant.tenants	Owning tenant.
name	VARCHAR(100)	NOT NULL	E.g. 'sa-deploy-prod'. Unique within org+tenant.
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active suspended deleted
description	TEXT	NULL	What this SA is used for.
created_by	UUID	NOT NULL FK → users(id) RESTRICT	Human who created this SA.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
last_active_at	TIMESTAMPTZ	NULL	Updated on each API key use.
UNIQUE(organization_id, tenant_id, name) · INDEX(tenant_id, status)			

identity.api_keys

Secrets for ServiceAccounts. Raw secret shown ONCE at creation — only bcrypt hash stored.

Column	Type	Constraints	Description
id	UUID	PK	
service_account_id	UUID	NOT NULL FK → service_accounts(id) CASCADE	Parent SA.
key_hash	VARCHAR(255)	NOT NULL	bcrypt hash. Original secret never stored.
key_prefix	VARCHAR(20)	NOT NULL	First 8 chars for display. E.g. 'ak_sc8fx2'.
scopes	VARCHAR[]	NOT NULL DEFAULT '{}'	Permission codes this key is limited to.
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active revoked expired
expires_at	TIMESTAMPTZ	NULL	Optional expiry. NULL = non-expiring.
last_used_at	TIMESTAMPTZ	NULL	Updated on each authenticated request.
created_by	UUID	NOT NULL FK → users(id) RESTRICT	Human who created this key.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
revoked_at	TIMESTAMPTZ	NULL	
INDEX(service_account_id) · INDEX(expires_at)			

identity.identity_realms

Registry of all Keycloak realms. One platform (shared), one operator (VPN), one per upgraded company org.

Column	Type	Constraints	Description
id	UUID	PK	
realm_name	VARCHAR(100)	UNIQUE NOT NULL	Keycloak realm name. Used as realm_ref everywhere.
realm_type	VARCHAR(20)	NOT NULL	platform org operator

organization_id	UUID	NULL FK-ref → tenant.organizations	NULL for platform/operator realms.
realm_url	VARCHAR(500)	NOT NULL	Full auth URL. E.g. acme.auth.platform.io
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active suspended deleted
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	When realm was provisioned.
UNIQUE(realm_name) · INDEX(organization_id) · INDEX(realm_type)			

identity.audit_log

Immutable append-only event store. DB-level trigger DENIES UPDATE and DELETE. Rows kept forever.

Column	Type	Constraints	Description
id	UUID	PK	
event_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Server time of event.
event_action	VARCHAR(100)	NOT NULL	E.g. 'user.registered', 'role_binding.revoked'.
actor_type	VARCHAR(20)	NOT NULL	user service_account platform_system
actor_id	UUID	NULL	user_id or SA id. NULL for system events.
target_type	VARCHAR(50)	NULL	What was acted on: 'user', 'role_binding', etc.
target_id	UUID	NULL	ID of the target entity.
tenant_id	UUID	NULL	Tenant context. NULL for platform events.
payload	JSONB	NULL	Event-specific data: {role, scope, reason, ...}.
ip_address	INET	NULL	Client IP at time of action.
correlation_id	UUID	NULL	X-Correlation-Id for distributed tracing.
INDEX(event_at DESC) · INDEX(actor_id) · INDEX(tenant_id, event_at DESC) · NO DELETE/UPDATE (trigger)			

Authorization Domain — 4 Tables

Answers: "Is this principal allowed to do this action here?" Schema: authz Runs after Identity has confirmed who you are.

authz.permissions

Global catalog of every possible action code. Codes are IMMUTABLE — never renamed or deleted. Deprecated codes are kept for audit history.

Column	Type	Constraints	Description
id	UUID	PK	
code	VARCHAR(100)	UNIQUE NOT NULL	Dot-separated: domain.resource.action. E.g. resource.action
scope_type	VARCHAR(20)	NOT NULL CHECK(IN('tenant','platform'))	Which scope this permission applies to.
description	TEXT	NULL	Human-readable. Shown in UI permission picker.
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active deprecated. NEVER deleted — soft-deprecated
deprecated_at	TIMESTAMPTZ	NULL	Set when status → deprecated.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
UNIQUE(code) · INDEX(scope_type, status) · CHECK: code matches '^[a-z]+(\.[a-z_]+)+\$'			

authz.roles

Named permission bundles. Unique within (scope_type, scope_id). is_system=true roles seeded at onboarding — API DELETE blocked.

Column	Type	Constraints	Description
id	UUID	PK	
name	VARCHAR(100)	NOT NULL	E.g. 'tenant-admin'. Unique per (scope_type, scope_id).
scope_type	VARCHAR(20)	NOT NULL	tenant platform
scope_id	UUID	NOT NULL FK-ref	tenant_id or operator_org_id depending on scope_type.
is_system	BOOLEAN	NOT NULL DEFAULT false	true = seeded at onboarding. Cannot delete via API.
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active deprecated. NEVER deleted.
description	TEXT	NULL	
created_by	UUID	NULL FK → identity.users(id) RESTRICT NULL for system-seeded roles.	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
UNIQUE(name, scope_type, scope_id) · INDEX(scope_id, scope_type, status)			

authz.role_permissions

M:N join table. Which permissions belong to which role. Rows CAN be deleted when a role's permission set changes.

Column	Type	Constraints	Description
role_id	UUID	PK, FK → authz.roles(id) ON DELETE CASCADE	Role that owns this permission.
permission_id	UUID	PK, FK → authz.permissions(id) ON DELETE RESTRICT	Permission being granted.
granted_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	When this mapping was added.

granted_by	UUID	NULL FK → identity.users(id) RESTRICT	Who added this. NULL for system seed.
PRIMARY KEY(role_id, permission_id) · INDEX(permission_id) for reverse lookup			

authz.user_roles

Assigns a role to a user (or SA) at a specific scope. Queried on EVERY API request. NEVER hard-deleted — revoked rows kept for audit.

Column	Type	Constraints	Description
id	UUID	PK	
user_id	UUID	NOT NULL FK-ref (polymorphic)	References identity.users OR identity.service_accounts (see
principal_type	VARCHAR(20)	NOT NULL DEFAULT 'user'	user service_account
role_id	UUID	NOT NULL FK → authz.roles(id) RESTRICT	Which role is being assigned.
scope_type	VARCHAR(20)	NOT NULL	Must match roles.scope_type.
scope_id	UUID	NOT NULL FK-ref	tenant_id or operator org_id. Must match role's scope_id.
status	VARCHAR(20)	NOT NULL DEFAULT 'active'	active revoked. NEVER deleted — revoked rows kept.
granted_by	UUID	NOT NULL FK → identity.users(id) RESTRICT	Tenant-admin who granted this.
granted_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
expires_at	TIMESTAMPTZ	NULL	Optional expiry. Scheduler auto-revokes when past.
revoked_by	UUID	NULL FK → identity.users(id) RESTRICT	Set when status → revoked.
revoked_at	TIMESTAMPTZ	NULL	Timestamp of revocation.
justification	TEXT	NULL	Optional reason. Good practice for audit clarity.
UNIQUE(user_id, role_id, scope_type, scope_id) INDEX(user_id, scope_type, scope_id, status) ← CRITICAL: hot path on every API request INDEX(scope_id, status) for listing roles in a tenant · INDEX(expires_at) for scheduler			

FK / Reference Sheet

All foreign key relationships. Hard FK = DB constraint. FK-ref = UUID stored, app-layer validation only.

From table.column	→ To table.column	Type	On Delete
credentials.user_id	users.id	HARD FK	CASCADE
sessions.user_id	users.id	HARD FK	CASCADE
service_accounts.created_by	users.id	HARD FK	RESTRICT
api_keys.service_account_id	service_accounts.id	HARD FK	CASCADE
api_keys.created_by	users.id	HARD FK	RESTRICT
roles.created_by	users.id	HARD FK	RESTRICT
role_permissions.role_id	roles.id	HARD FK	CASCADE
role_permissions.permission_id	permissions.id	HARD FK	RESTRICT
role_permissions.granted_by	users.id	HARD FK	RESTRICT
user_roles.role_id	roles.id	HARD FK	RESTRICT
user_roles.granted_by	users.id	HARD FK	RESTRICT
user_roles.revoked_by	users.id	HARD FK	RESTRICT
service_accounts.organization_id	tenant.organizations.id	FK-ref	app-layer
service_accounts.tenant_id	tenant.tenants.id	FK-ref	app-layer
identity_realms.organization_id	tenant.organizations.id	FK-ref	app-layer
roles.scope_id	tenant.tenants.id (or op-org)	FK-ref (polymorphic)	app-layer
user_roles.scope_id	tenant.tenants.id (or op-org)	FK-ref (polymorphic)	app-layer
user_roles.user_id	users.id OR service_accounts.id	FK-ref (polymorphic)	app-layer

Schema Design Decisions & Explanations

1. Roles are never stored in the JWT token

The JWT contains only: sub (keycloak_subject), iss (realm URL), exp (expiry), sid (session id), typ. Roles are resolved from authz.user_roles on every API request. This means: if carol's role is revoked at 10:00, her 10:01 request returns 403 — even if her JWT does not expire until 10:15. Without this design, you must wait up to the full JWT TTL for access to be truly removed.

2. user_roles rows are never hard-deleted

Revocation sets status='revoked' and records revoked_at and revoked_by. The row stays forever. This is essential for compliance: you must answer 'did alice have tenant-admin access at 09:58 on 15-Mar-2024?' Hard-deleting would make that question unanswerable. Archived rows also show who granted access and why (justification field).

3. permission codes are immutable

Codes like 'resource.allocation.approve' are permanent identifiers. If a permission is retired, status='deprecated' but the row and code remain. Audit log entries reference actions by their string code. Deleting a permission record would make old audit entries uninterpretable.

4. user_roles.user_id is polymorphic

Both identity.users (humans) and identity.service_accounts (robots) can hold RBAC roles. The principal_type column ('user' or 'service_account') tells the app which table to use. This avoids two separate user_roles and sa_roles tables with identical structure. There is intentionally no hard FK here — the app validates which table to look in.

5. FK-refs across domain boundaries

authz.user_roles.scope_id references tenant.tenants.id — but this is NOT a hard FK in the DB. The Authorization domain does not own Tenant tables. A hard FK would mean Tenant migrations must coordinate Authorization schema changes — breaking domain independence. Instead, scope_id is validated at the application layer on every write.

6. scope_id is always required in RBAC queries

Every authorization check MUST include both user_id AND scope_id from X-Tenant-Id header. Query: WHERE user_id=? AND scope_type='tenant' AND scope_id=? AND status='active'. Omitting scope_id would allow cross-tenant permission leakage: alice's tenant-admin role in acme-prod could match an acme-dev check.

7. audit_log is append-only at DB level

A PostgreSQL trigger DENIES any UPDATE or DELETE on identity.audit_log. Only INSERT is allowed — from the app service account. Even a compromised platform-admin cannot delete their audit trail. This makes audit records tamper-evident by design, not just policy.

8. identity_realms is a DB table, not just config

The platform tracks every Keycloak realm as a DB record. JWT validation middleware checks: 'is the iss claim in our realms registry, and what type?' before any RBAC check. It also makes realm creation auditable and enables realm-level operations (suspend org realm, list all org realms, etc). The realm_type field is what enforces the platform scope wall: iss must = operator realm type for any /admin route — checked before any DB query.

End of Identity & Authorization Schema Reference — TenantPlatform v2